

Kruskal's Algorithm

Date / /

Page

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
typedef struct
```

```
int u, v, w;
```

```
} Edges;
```

```
Edges edges[100]; result[100];
```

```
int parent[100];
```

```
int find (int i) {
```

```
    while (parent[i] != i)
```

```
        i = parent[i];
```

```
    return i;
```

```
}
```

```
void union (int i, int j) {
```

```
    parent[i] = j;
```

```
}
```

```
void sortEdges (int e) {
```

```
    int i, j;
```

```
    Edge temp;
```

```
    for (i = 0; i < e - 1; i++) {
```

```
        for (j = 0; j < e - i - 1; j++) {
```

```
            if (edges[j].w > edges[j+1].w) {
```

```
                temp = edges[j];
```

```
                edges[j] = edges[j+1];
```

```
                edges[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```

int kruskal ( Edge edges[], int n, int m) {
    int i, v, count = 0, cost = 0;
    for (i = 0; i < n; i++)
        parent[i] = i;
    sort(edges, m);
    for (i = 0; i < m; i++) {
        count++;
        v = find(edges[i].u);
        v = find(edges[i].v);
        if (v != v) {
            result[count-1] = edges[i];
            cost += edges[i].w;
            union(v, v);
        }
    }
    printf("Minimal Spanning Tree: \n");
    for (i = 0; i < count; i++) {
        printf("%d - %d \t %d \n", result[i].u,
            result[i].v, result[i].w);
    }
    printf("Total cost = %d \n", cost);
}

```

```

int main() {
    int n, m;
    Edges edges[100];
    printf("Enter the no. of vertices: ");
    scanf("%d", &n);
    printf("Enter the no. of edges: ");
    scanf("%d", &m);
    printf("Enter edges: \n");
    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u,
            &edges[i].v, &edges[i].w);
    }
}

```

```

clock = clock(), end;
start = clock();
kruskal (edges, n, m);
end = clock();
double time = (double)(end - start) /
                CLOCKS_PER_SEC;
printf ("Execution time : %f", time);
return 0;

```

y

Output .

Enter no. of vertices: 9

Enter no. of edges : 14

Enter edges:

0 1 4	3 4 9
0 7 8	3 5 14
1 2 8	4 5 20
1 7 11	5 6 2
2 3 7	6 7 1
2 8 2	6 8 6
2 5 4	7 1 7

Minimum spanning tree

6-7 1

8-2 2

5-6 2

0-1 4

2-5 4

2-3 7

7-1 7

3-4 9

Total cost = 3

Execution time : 0.00100

